

Individual Contribution Reflection

Raven Ge (500133392)

COMP4347 / COMP5347 Assignment 2 - MERN Quiz Game

Subsystem: Quiz Flow, Scoring, Review Mode, History, and Leaderboard

1. Subsystem ownership. I owned the player quiz subsystem across backend and frontend: random quiz start, sequential answer flow, final scoring, Review Mode payloads, history, leaderboard, and the player-facing UI/UX around those states. This subsystem connects React quiz state to Express quiz routes and MongoDB Score/Question records.

2. Major technical challenge. The major challenge was keeping the quiz efficient and consistent after Review Mode was introduced. The first scoring approach risked repeated Question lookups and review drift. I refactored scoring around a single batch query and a question map, then reused that data to build review[] with the selected answer, correct answer, topic, and explanation. Later option shuffling made the visible selected index important, so the review experience had to match what the user actually saw during the attempt.

3. Mermaid sequence diagram.

```
sequenceDiagram
    participant Player as Player Browser
    participant Quiz as React Quiz Flow
    participant API as Quiz Controller
    participant Q as Question Model
    participant S as Score Model
    Player->>Quiz: Start quiz and answer sequentially
    Quiz->>API: GET /api/quiz/start
    API->>Q: sample active questions
    API-->>Quiz: 10 public questions
    Quiz->>API: POST /api/quiz/submit answers
    API->>Q: batch fetch questions
    API-->>S: save score and full answers
    API-->>Quiz: score, scoreId, review[]
    Quiz-->>Player: result and Review Mode
```

4. Review Mode design reflection. Review Mode was implemented as a post-completion learning flow, not a scoring change. I used server-side review assembly so the final screen and history page can show selected answer, correctness, correct answer, and explanation from the saved attempt. The UI design favors learning: wrong answers are surfaced clearly, history enables re-review, and leaderboard remains separate from explanation feedback.

5. Cross-system understanding. This subsystem depends on the full MERN stack: React collects input and displays feedback, Express exposes protected REST routes, middleware enforces role and validation rules, Mongoose persists the relevant state, and the shared response envelope keeps frontend error handling predictable.

6. Git commit history analysis. The table below cites meaningful commits from the GitHub Enterprise history and explains what each contributed technically.

Commit hashes: def48eb 62ca283 8724146 e22b914 4c1730d 268d0c6 41f7ce8 d17fe3c 657d15b 0102a29 d04b018 93a9096 64d01cf c5ea913

Hash	Focus	Evidence
def48eb	Quiz API	Reworked quiz controller, batch scoring, review assembly, history and leaderboard.
62ca283	Quiz frontend	Built early QuizContext, quiz page, review page, and history flow.
8724146	UI system	Major gameplay and Review Mode UI overhaul.
e22b914	Assets and interactions	Added background assets and fixed QuizScreens interactions.
4c1730d	Flow polish	Refined quiz animation and interaction logic.
268d0c6	Review UI	Updated quiz logic, Review Mode layout, and history styling.
41f7ce8	Question logic	Added shuffleQuestion utilities and updated quiz/question logic.
d17fe3c	Player flow fixes	Fixed QuizScreens, HistoryPage, login/register UI integration.
657d15b	Admin/player boundary	Added admin quiz block middleware and UI route effects.
0102a29	Review archive	Expanded review cards, formatting utilities, and history/review pages.
d04b018	Navigation	Added navigation system and gameplay HUD integration.
93a9096	Code quality	Cleaned CSS and quiz UI quality issues.
64d01cf	Landing/admin route	Fixed landing page and admin route protection.
c5ea913	Leaderboard	Implemented best-score leaderboard tie-breaking and cap.

7. Final reflection. My contribution was not only a list of commits; it was a bounded subsystem responsibility with evidence. The commits above show implementation, validation, integration, and final submission hardening. In the demo I can explain both my own files and how my subsystem interacts with authentication, quiz attempts, admin data, Review Mode, and MongoDB persistence.